ARQUITECTURA DE AGENTES IA · UTDT 2026

Módulo 4
Tool Use y MCP

MCP · Principle of Least Power · Tool Design · ADK2 Tools

UTDT 2026 | MÓDULO 4 DE 13

AGENDA · MÓDULO 4

Lo que vamos a cubrir hoy

BLOQUE 1

- ¿Qué es un tool y por qué importa?
- Tres tipos de tools
- Tool metadata — el contrato invisible

BLOQUE 2

- MCP: el estándar universal
- Arquitectura MCP Server / Client
- Seguridad y brechas activas

BLOQUE 3

- Principle of Least Power
- ADK2 en Vertex AI
- Automated Tool Development

CONCEPTO FUNDAMENTAL

Los tools son la capa de ejecución

> *"Un modelo de lenguaje sin tools es un oráculo — responde pero no actúa. Con tools, se convierte en un agente que modifica el mundo."*

SIN TOOLS

- Genera texto
- Conocimiento estático (cutoff)
- No puede leer ni escribir
- No tiene acceso a APIs

CON TOOLS

- Ejecuta código, llama APIs
- Datos en tiempo real
- Lee y modifica bases de datos
- Orquesta otros servicios

El diseño de los tools determina si el agente es útil o peligroso.

TAXONOMÍA · TRES TIPOS DE TOOLS

Local · API-based · MCP

LOCAL TOOLS

- Funciones puras co-deployed con el agente
- Matemáticas, fechas, calendarios, mapas
- Predecibles y precisas
- Sin dependencias externas

API-BASED TOOLS

- Llamadas HTTP a servicios externos
- Datos en tiempo real, escalabilidad
- Requieren manejo de errores
- Rate limits y autenticación

MCP TOOLS

- Model Context Protocol
- JSON-RPC 2.0 estandarizado
- "USB-C para IA"
- Cualquier agente, cualquier servidor

El metadata es tan importante como la lógica

El modelo de lenguaje nunca ve el código del tool — solo ve su nombre, descripción y schema.

REGLA 1 — NOMBRE PRECISO

Nombres demasiado generales hacen que el LLM llame al tool cuando no corresponde.

✗ `execute_query`

✓ `get_user_profile_by_id`

REGLA 2 — DESCRIPCIÓN CLARA

Descripciones superpuestas generan confusión en la selección del tool.

Cada tool debe describir exactamente cuándo usarlo y cuándo NO.

REGLA 3 — SCHEMA ESTRICTO

Schemas explícitos de input/output reducen los misfires del modelo.

Tipos, rangos, enums — todo especificado, nada ambiguo.

`@tool` decorator y `.bind_tools()`

```python
from langchain_core.tools import tool

@tool
def multiply(x: float, y: float) -> float:
    """Multiply 'x' times 'y'. Use ONLY for numeric multiplication.
    Do not use for string concatenation or list operations."""
    return x * y

# Adjuntar el schema del tool al modelo
llm_with_tools = llm.bind_tools([multiply])

# El modelo decide si invocar el tool
ai_msg = llm_with_tools.invoke(messages)

# Ejecutar los tool_calls que el modelo solicitó
for tool_call in ai_msg.tool_calls:
    tool_msg = multiply.invoke(tool_call)
```

@tool — Registra la función y auto-genera el JSON Schema desde type hints y docstring.

.bind_tools() — Adjunta los schemas al modelo para que pueda seleccionarlos e invocarlos.

El "USB-C" para herramientas de IA

> *"Antes de MCP, cada agente necesitaba adaptadores custom para cada servicio. MCP estandariza la interfaz — implementás el servidor una vez, cualquier agente lo puede usar."*

- Estándar abierto sobre JSON-RPC 2.0
- Un servidor expone métodos (`listFiles`, `runAnalysis`…)
- Cualquier cliente MCP puede descubrir y llamar esos métodos
- Transportes: HTTPS/WebSocket (remoto) o STDIO (local)

SIN MCP VS. CON MCP

N agentes × M servicios = N×M adaptadores

Con MCP:

N agentes + M servidores MCP = N+M implementaciones

MCP Server y MCP Client

MCP SERVER

- Proceso que expone métodos vía JSON-RPC 2.0
- Catalogo de tools: `tools/list`
- Ejecución: `tools/call`
- Puede correr local (STDIO) o remoto (HTTPS)
- Ejemplo: servidor que expone BigQuery, Sheets, Drive

MCP CLIENT (EL AGENTE)

- Solicita el catálogo de métodos disponibles
- El LLM selecciona el tool adecuado
- Invoca el tool y procesa la respuesta
- Compatible con Claude, Gemini, GPT-4, ADK2
- Ejemplo: agente Vertex AI conectado al servidor GCP

Catálogos públicos de servidores MCP: mcp.so y Glama.ai — repositorios con cientos de endpoints listos para usar.

STDIO vs. HTTPS/WebSocket

STDIO — LOCAL

- El servidor corre como subproceso del agente
- Comunicación vía stdin/stdout
- Latencia mínima, sin overhead de red
- Ideal para desarrollo y tools de filesystem
- Ejemplo: MCP para manipular archivos locales

HTTPS / WEBSOCKET — REMOTO

- Servidor independiente accesible vía red
- Múltiples agentes conectados simultáneamente
- Requiere TLS + autenticación
- Ideal para servicios compartidos en producción
- Ejemplo: MCP server en Cloud Run + Vertex AI agents

La comparación que importa

	CUSTOM ADAPTER	MCP
Setup	Código bespoke por servicio	Un servidor MCP único
Reutilización	Re-implementar por agente	Cualquier agente consume el mismo server
Mantenimiento	Frágil, se multiplica con servicios	Actualizás el server una sola vez
Seguridad	Control total	Brechas en access control (investigación activa)
Adopción	Inmediata	Requiere implementar el server

> El costo de implementar MCP se amortiza a partir del segundo agente que reutiliza el mismo servidor. En organizaciones con múltiples agentes, MCP es la opción que escala: el adapter bespoke no lo hace.

Brechas activas que hay que conocer

AUTENTICACIÓN

MCP no estandariza autenticación todavía. Cada implementación resuelve esto por su cuenta — riesgo de servidores públicos sin auth.

ACCESS CONTROL

Los permisos granulares por tool/por usuario no están en la especificación base. El servidor debe implementarlos explícitamente.

PAYLOAD INJECTION

Un payload malicioso en la respuesta de un tool puede instruir al agente a realizar acciones no autorizadas — vector de prompt injection.

Estado actual: Área de investigación activa. No asumas que "MCP es seguro por default" — implementá validación en cada capa.

Cuándo NO usar MCP

¿QUIÉN LO OPERA?

Un servidor MCP es un servicio productivo más: tiene dueño, guardia, SLA y presupuesto. Si nadie se lo apropia, queda como un adaptador huérfano con un protocolo estándar.

La pregunta previa a elegir MCP no es técnica: es de qué equipo es.

VERSIONADO DEL CATÁLOGO

El descubrimiento automático corta en los dos sentidos: si un tool cambia de comportamiento sin cambiar de nombre, todos los agentes conectados usan la versión nueva en la sesión siguiente. Sin deploy, sin aviso, sin revisión.

El catálogo necesita versión explícita, cambios aditivos y ventana de deprecación — como cualquier API pública.

QUÉ PIDE AUDITORÍA

- Quién autentica al cliente, y con qué identidad queda registrada la llamada
- Permisos por tool y por usuario, no por servidor
- Traza de cada `tools/call`: quién, qué, con qué argumentos
- Qué pasa con el dato que sale hacia el modelo

Soporte de tools por plataforma

PLATAFORMA	ACCESO A TOOLS	NOTAS
Anthropic (Claude)	Completo vía Messages API	API-first; funciona en Bedrock y Vertex
Google (Gemini)	FunctionCallingConfig vía Vertex AI	Integración nativa con GCP services
OpenAI (GPT-4)	function_calling / tool_calls	Plug-ins solo en ChatGPT UI, no API pública
Microsoft (Phi)	Azure AI Foundry endpoints	Integración estrecha con servicios Azure

> En Vertex AI Agent Builder, Gemini accede a tools mediante `FunctionCallingConfig` y extensiones nativas para BigQuery, Cloud SQL, Search, y otros servicios GCP — sin código de integración adicional.

Tools en ADK2 — Tres formas de definirlas

```python
# Forma 1: FunctionTool — función Python estándar
from google.adk.tools import FunctionTool

def get_weather(city: str, unit: str = "celsius") -> dict:
    """Retrieve current weather for a given city.
    Args:
        city: Name of the city (e.g. 'Buenos Aires')
        unit: Temperature unit — 'celsius' or 'fahrenheit'
    Returns:
        dict with temp, humidity, conditions
    """
    # implementación...
    return {"temp": 22, "unit": unit, "conditions": "sunny"}

weather_tool = FunctionTool(func=get_weather)
```

ADK2 extrae automáticamente el schema JSON desde los type hints y el docstring. La calidad del docstring impacta directamente en cuándo el agente elige usar el tool.

AgentTool y MCPToolset

```python
from google.adk.tools import AgentTool, MCPToolset
from google.adk.tools.mcp_tool.mcp_toolset import StdioServerParameters

# AgentTool — un sub-agente como tool
research_agent = Agent(name="researcher", model="gemini-2.5-flash", ...)
research_tool = AgentTool(agent=research_agent)

# MCPToolset — conectar a servidor MCP local (STDIO)
mcp_tools = MCPToolset(
    connection_params=StdioServerParameters(
        command="python",
        args=["-m", "my_mcp_server"]
    )
)

# Agente raíz con ambos tipos de tools
root_agent = Agent(
    name="orchestrator",
    model="gemini-2.5-flash",
    tools=[weather_tool, research_tool, mcp_tools]
)
```

FunctionTool con BigQuery en Vertex AI Agent Builder

```python
from google.adk.tools import FunctionTool
from google.cloud import bigquery

def query_sales_data(
    region: str,
    start_date: str,
    end_date: str,
    metric: str = "revenue"
) -> dict:
    """Query aggregated sales data from the data warehouse.
    Use ONLY for questions about sales performance, revenue,
    and regional metrics. Do NOT use for user profile data.
    Args:
        region: GCP region code (e.g. 'southamerica-east1')
        start_date: ISO date string 'YYYY-MM-DD'
        end_date: ISO date string 'YYYY-MM-DD'
        metric: 'revenue' | 'units' | 'margin'
    """
    # `metric` es identificador de columna, no valor: no se liga → allowlist
    if metric not in {"revenue", "units", "margin"}:
        return {"error": f"metric no permitido: {metric}"}
    client = bigquery.Client()
    query = f"""
        SELECT {metric}, SUM(value) as total
        FROM `project.dataset.sales`
        WHERE region = @region
        AND date BETWEEN @start_date AND @end_date
        GROUP BY {metric}
    """
    # Los VALORES sí van ligados — previene SQL injection
    job_config = bigquery.QueryJobConfig(
        query_parameters=[
            bigquery.ScalarQueryParameter("region", "STRING", region),
            bigquery.ScalarQueryParameter("start_date", "DATE", start_date),
            bigquery.ScalarQueryParameter("end_date", "DATE", end_date),
        ]
    )
    results = client.query(query, job_config=job_config).result()
    return {"rows": [dict(row) for row in results]}

sales_tool = FunctionTool(func=query_sales_data)
```

Conectar ADK2 a un servidor MCP en Cloud Run

```python
from google.adk.tools import MCPToolset
from google.adk.tools.mcp_tool.mcp_toolset import (
    StreamableHTTPConnectionParams
)

# MCPToolset remoto — servidor MCP en Cloud Run
gcp_mcp_tools = MCPToolset(
    connection_params=StreamableHTTPConnectionParams(
        url="https://my-mcp-server-xyz.run.app/mcp",
        headers={
            "Authorization": f"Bearer {get_identity_token()}",
            "Content-Type": "application/json"
        }
    )
)

# El agente descubre automáticamente los tools disponibles
# en el servidor MCP remoto (tools/list)
agent = Agent(
    name="gcp_agent",
    model="gemini-2.5-flash",
    tools=[gcp_mcp_tools],
    instruction="You are a GCP operations agent..."
)
```

Las tools que modifican estado persistente

¿QUÉ SON?

- Tools que escriben en bases de datos, archivos, colas de mensajes
- Tools que envían emails, notificaciones, transacciones financieras
- Tools que llaman APIs con side effects (POST, PUT, DELETE)

EL CASO REAL QUE MOTIVÓ EL PRINCIPLE OF LEAST POWER

Un agente de optimización de base de datos recibió acceso completo a SQL. "Optimizó el rendimiento" eliminando la mitad de las filas que consideró redundantes. Las filas eran datos de producción.

SEÑALES DE ALERTA

- Tool llamado con alta frecuencia
- Parámetros fuera de rango esperado
- Secuencias de operaciones inusuales
- Invocaciones fuera de horario laboral

Principle of Least Power

> *"Un agente solo debe poder hacer exactamente lo que necesita para su tarea — ni una operación más."*

SCOPE ESTRICTO

- Registrar solo operaciones específicas
- `get_user_profile`, no `execute_sql`
- Read-only nunca tiene write/delete
- Separar agentes por nivel de privilegio

INPUTS SEGUROS

- Sanitizar todos los inputs
- Parámetros ligados (binding) — no interpolación
- Validar tipos, rangos, enums
- Rechazar inputs que no matcheen el schema

OBSERVABILIDAD

- Loggear cada invocación
- Alertar sobre patrones anómalos
- Retener historial de ejecuciones
- Circuit breaker para anomalías

De SQL libre a tool con scope mínimo

Anti-pattern — SQL libre:

```python
@tool
def execute_database_query(sql: str) -> list:
    """Execute any SQL query."""
    # El agente puede: SELECT, DROP, UPDATE...
    # Un prompt injection puede borrar tablas
    return db.execute(sql)
```

Pattern correcto — scope mínimo:

```python
@tool
def get_active_users(
    department: str,
    limit: int = 100
) -> list:
    """Get active users filtered by department.
    Max 100 results. Read-only operation."""
    # SQL hardcoded, params ligados
    return db.execute(
        "SELECT id, name FROM users "
        "WHERE dept = ? AND active = 1 "
        "LIMIT ?",
        (department, min(limit, 100))
    )
```

Checklist de seguridad para API-based tools

TRANSPORTE Y AUTH

- HTTPS obligatorio en todas las llamadas externas
- Autenticación fuerte (OAuth2, API keys con rotación)
- Tokens con tiempo de expiración corto

RESILIENCIA

- Rate limit handling con backoff exponencial
- Fallback a proveedor alternativo
- Timeout agresivos — no bloquear el agente

PRIVACIDAD

- Anonimizar datos sensibles antes de enviar
- No loggear PII en registros de invocación
- Cumplimiento con GDPR / normativa local

Tools preconstruidas: ventajas y límites

¿QUÉ SON?

Módulos prebuilt de OpenAI, Anthropic, Google, Microsoft. Integración rápida para casos estándar: búsqueda web, generación de código, análisis de imágenes.

VENTAJAS

- Integración en minutos vs. días
- Mantenidos por el proveedor
- Testeados en producción a escala

LÍMITES

- Menos customización posible
- Dependencia de vendor
- Caja negra — difícil debuggear

En Vertex AI Agent Builder, las extensiones built-in (Google Search, Code Interpreter, Vertex AI APIs) son plug-in tools gestionadas por Google.

Foundation Models como Tool Makers

EL PROCESO

1. Darle al LLM el spec OpenAPI o ejemplos de código
2. El modelo redacta los wrappers de función
3. Ejecutar en sandbox — el modelo critica su propio output
4. Revisión humana antes de CI/CD

CUÁNDO USARLO

- APIs con muchos endpoints (más de 50 operaciones)
- APIs que evolucionan frecuentemente
- Mantener wrappers sincronizados con cambios
- Prototipado rápido de integraciones nuevas

> *"El modelo que usa los tools puede generar los tools — cerrando el ciclo de desarrollo. La revisión humana sigue siendo obligatoria antes de producción."*

Agentes que escriben sus propios tools en runtime

¿CÓMO FUNCIONA?

El agente encuentra una API desconocida durante la ejecución, genera el wrapper de código necesario, lo ejecuta en un sandbox aislado, y lo usa para completar la tarea.

REQUISITOS DE SEGURIDAD

- Sandbox obligatorio — código no revisado no toca producción
- Timeouts estrictos de ejecución
- Sin acceso a red desde el sandbox
- Revisión de output antes de usar el resultado

Vertex AI Code Interpreter es el ejemplo canónico: el agente genera y ejecuta código Python en un entorno aislado gestionado por Google.

Lo que nunca hay que hacer

SQL LIBRE

Exponer `execute_database_query(sql: str)` permite que prompt injection borre tablas enteras de producción.

SIN ERROR HANDLING

Si un API tool falla sin manejo de errores, el agente completo se rompe. Los servicios externos tienen downtime — el agente no puede.

TOOLS SIN REVISIÓN

Tools generados automáticamente que van directo a producción sin revisión humana. El LLM puede generar código con bugs sutiles o vectores de seguridad.

Anti-pattern adicional: Nombres de tools demasiado genéricos. `execute_database_query` → el LLM lo usa para TODO, incluyendo contextos donde no corresponde.

CONEXIONES · MÓDULOS RELACIONADOS

Tool Use en el contexto del curso

← MÓDULO 2

Los tres tipos de tools se presentaron como categorías. Aquí los profundizamos con implementación y seguridad.

→ MÓDULO 5

Orquestación — cómo el agente selecciona cuál tool invocar y en qué orden. Tool selection como parte del ciclo de razonamiento.

→ MÓDULO 12

Seguridad — prompt injection en tools stateful, ataques vía payload MCP, y defensa en profundidad para agentes con acceso a datos reales.

Módulo 8: Protocolo A2A (Agent-to-Agent) — el equivalente a MCP pero para comunicación entre agentes, no entre agente y tool.

SÍNTESIS

Cinco ideas que se llevan del Módulo 4

1 El metadata del tool (nombre, descripción, schema) controla con qué precisión el LLM lo selecciona — es tan crítico como la lógica.

2 MCP es el estándar emergente para interoperabilidad de tools — implementás el servidor una vez, cualquier agente lo reutiliza.

3 Las stateful tools son la categoría de mayor riesgo: Principle of Least Power, scope mínimo, logging de toda invocación.

4 Los Foundation Models pueden generar tools desde API specs — acelerando el desarrollo con revisión humana obligatoria como gate.

5 Las API tools requieren: error handling, rate limit awareness, autenticación robusta y fallback a proveedores alternativos.

PRÓXIMA CLASE · PREVIEW

Módulo 5 — Orquestación de Agentes

Tenemos los tools. Ahora la pregunta es: ¿cómo decide el agente cuál invocar, en qué orden, y qué hacer con el resultado?

- El ciclo Percibir → Razonar → Actuar aplicado a tool selection
- Planificación: ReAct, Chain of Thought, Tree of Thought
- Manejo de dependencias entre tool calls
- Paralelización de tools independientes en ADK2
- Error recovery — qué hace el agente cuando un tool falla

PARA LA PRÓXIMA CLASE

- Completar el lab del Módulo 4
- Leer sobre el patrón ReAct
- Explorar el ADK2 sequential agent example